COMPTE RENDU

BAKAN Aymen

CONTEXTE ET EXISTANT

La chaîne MediaTek86 dispose de plusieurs médiathèques gérant des documents de type livres, DVD et revues. Une application de bureau en C# existe déjà, exploitant une API REST en PHP pour l'accès à une base de données MySQL. L'objectif de cet atelier est de faire évoluer cette application en ajoutant de nouvelles fonctionnalités.

Langages et technologies utilisés :

- C# (.NET Framework 4.7.2) — application de bureau
- PHP — API REST
- MySQL — base de données relationnelle
- Visual Studio 2022
- NetBeans
- WampServer
- GitHub

MISSION 2 — TÂCHE 1 : Gérer les commandes de livres et DVD

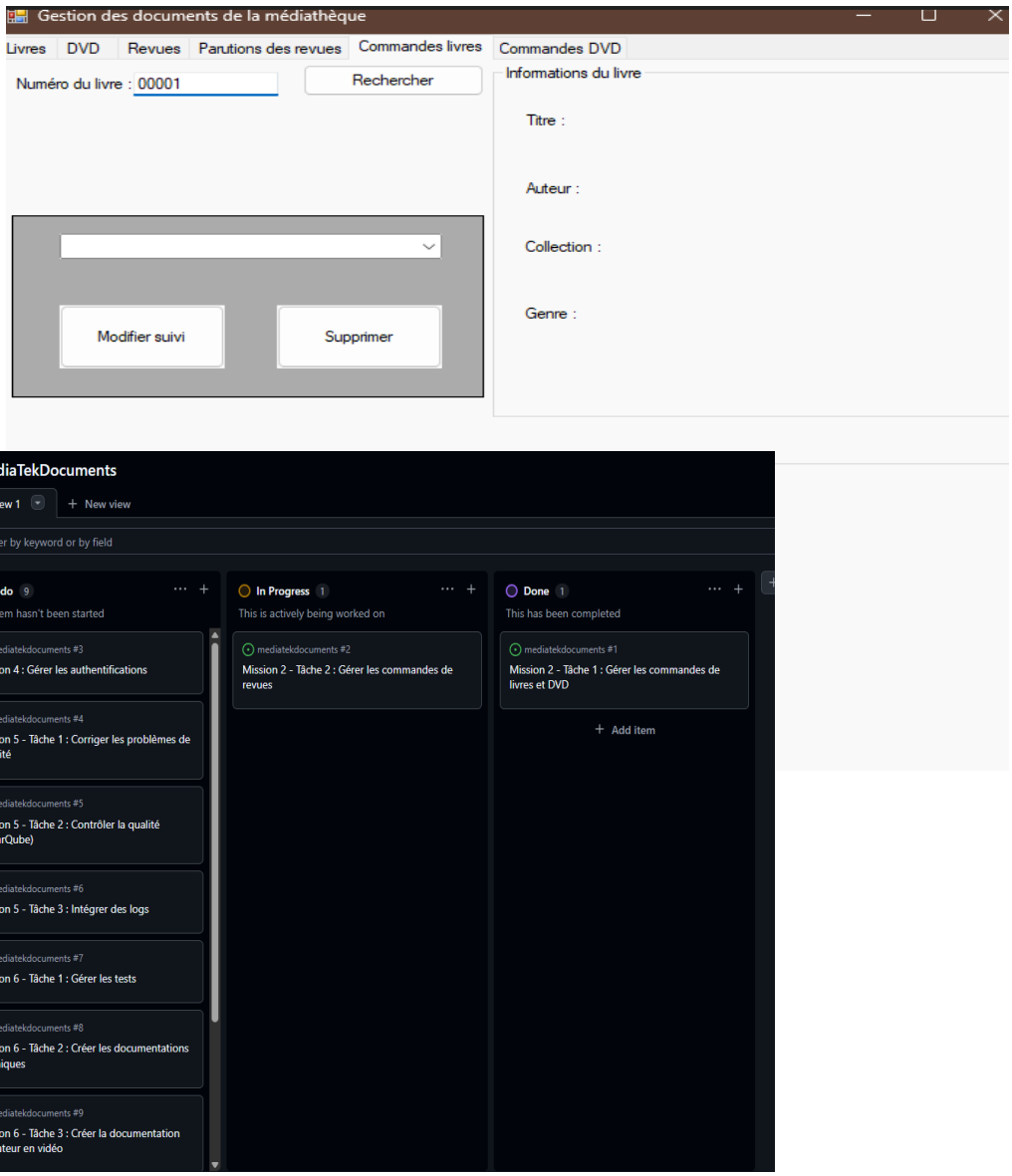
Demande : Créer un onglet permettant de gérer les commandes de livres et DVD. L'onglet doit permettre de rechercher un livre par son numéro, afficher ses informations et la liste de ses commandes, ajouter une nouvelle commande, modifier l'étape de suivi et supprimer une commande non livrée.

Diagramme de cas d'utilisation : L'utilisateur peut : rechercher un livre, consulter ses commandes, ajouter une commande, modifier le suivi d'une commande, supprimer une commande.

MCD — modifications apportées à la BDD : Ajout de la table suivi (id, libelle) reliée à commandedocument via la clé étrangère idSuivi. Ajout de 2 triggers : un pour créer les exemplaires lors de la livraison, un pour supprimer en cascade.

Classes impactées :

- Suivi.cs — nouvelle classe modélisant une étape de suivi
- CommandeDocument.cs — nouvelle classe modélisant une commande de document
- Access.cs — ajout des méthodes GetAllSuivis(), GetCommandesDocument(), AddCommandeDocument(), UpdateSuiviCommandeDocument(), DeleteCommandeDocument()
- FrmMediatekController.cs — ajout des mêmes méthodes en délégation
- FrmMediatek.cs — ajout de l'onglet Commandes livres avec toute la logique
- MyAccessBDD.php — ajout des requêtes SQL correspondantes



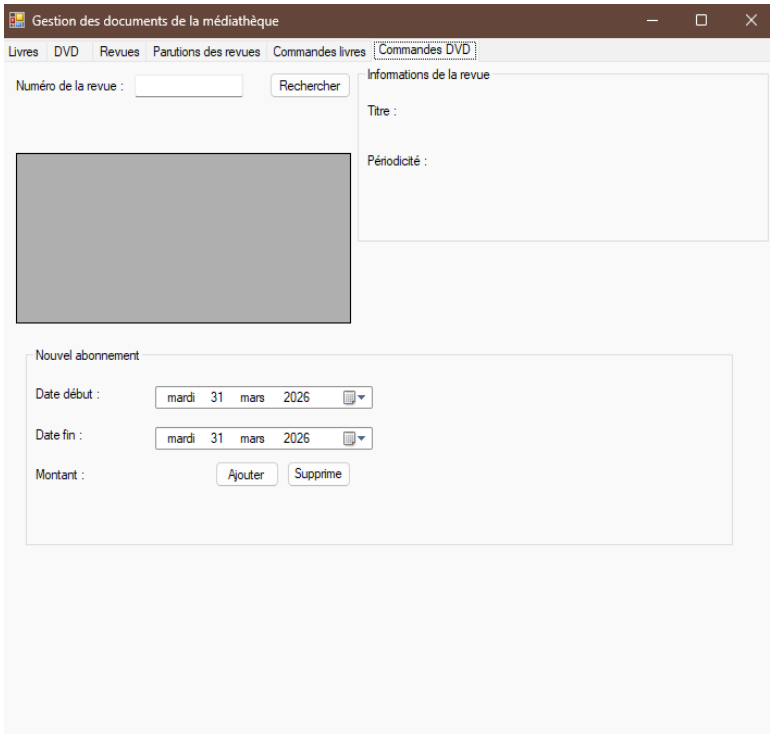
MISSION 2 — TÂCHE 2 : Gérer les commandes de revues
Demande : Créer un onglet permettant de gérer les abonnements aux revues. L'onglet permet de rechercher une revue par son numéro, afficher ses informations et la liste de ses abonnements, ajouter un nouvel abonnement et supprimer un abonnement.

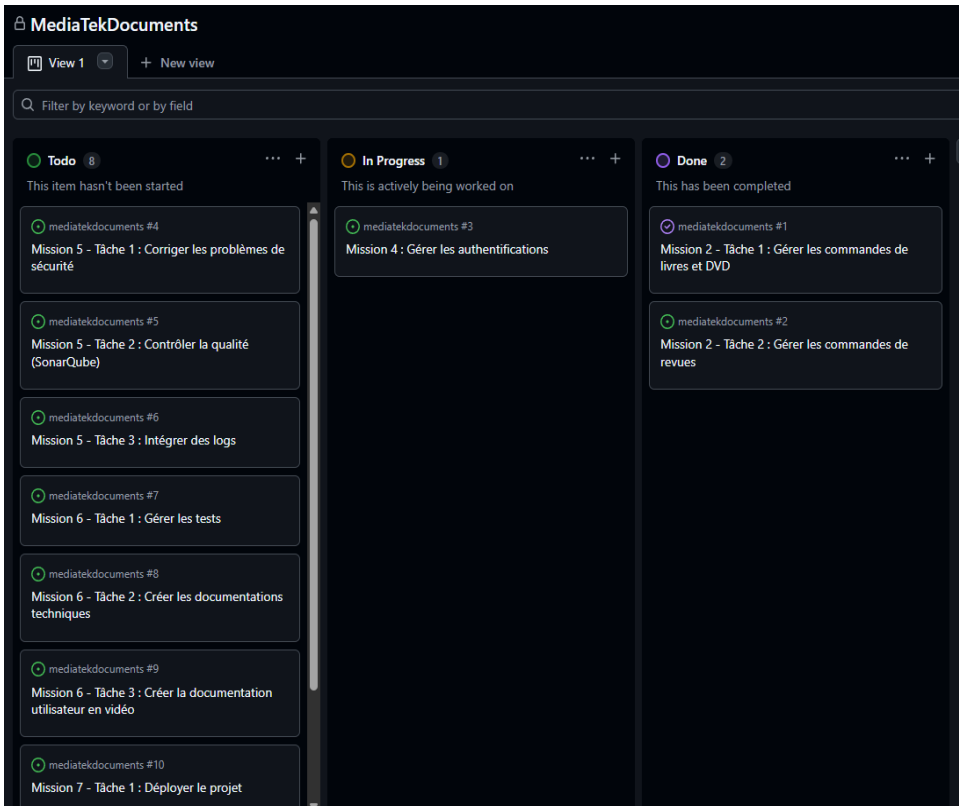
Diagramme de cas d'utilisation : L'utilisateur peut : rechercher une revue, consulter ses abonnements, ajouter un abonnement, supprimer un abonnement.

MCD — modifications apportées à la BDD : Ajout des colonnes dateCommande et montant dans la table abonnement existante.

Classes impactées :

- Abonnement.cs — nouvelle classe modélisant un abonnement
- Access.cs — ajout des méthodes GetAbonnementsRevue(), AddAbonnement(), DeleteAbonnement()
- FrmMediatekController.cs — ajout des mêmes méthodes en délégation
- FrmMediatek.cs — ajout de l'onglet Commandes revues avec toute la logique
- MyAccessBDD.php — ajout des requêtes SQL correspondantes





MISSION 4 — Gérer les authentifications
Demande : Ajouter une fenêtre d'authentification au démarrage de l'application. Selon le service de l'utilisateur connecté, limiter les accès aux différents onglets. Le service Culture n'a aucun accès et l'application se ferme avec un message. Le service Comptabilité a

accès aux commandes uniquement.

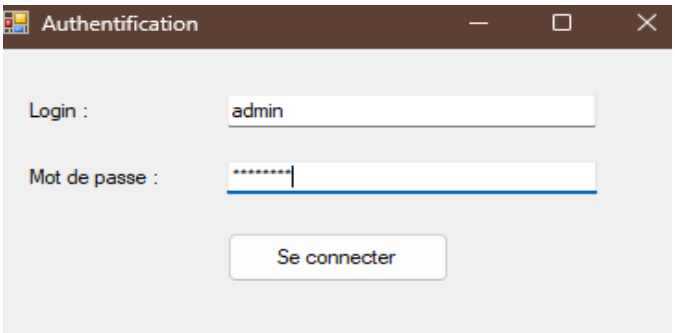
Diagramme de cas d'utilisation : L'utilisateur peut : se connecter avec login/mdp, accéder aux fonctionnalités selon son service, voir un message d'erreur si droits insuffisants.
MCD — modifications apportées à la BDD : Ajout de la table service (id, libelle) et de la table utilisateur (id, nom, prenom, login, mdp, idService) reliée à service.

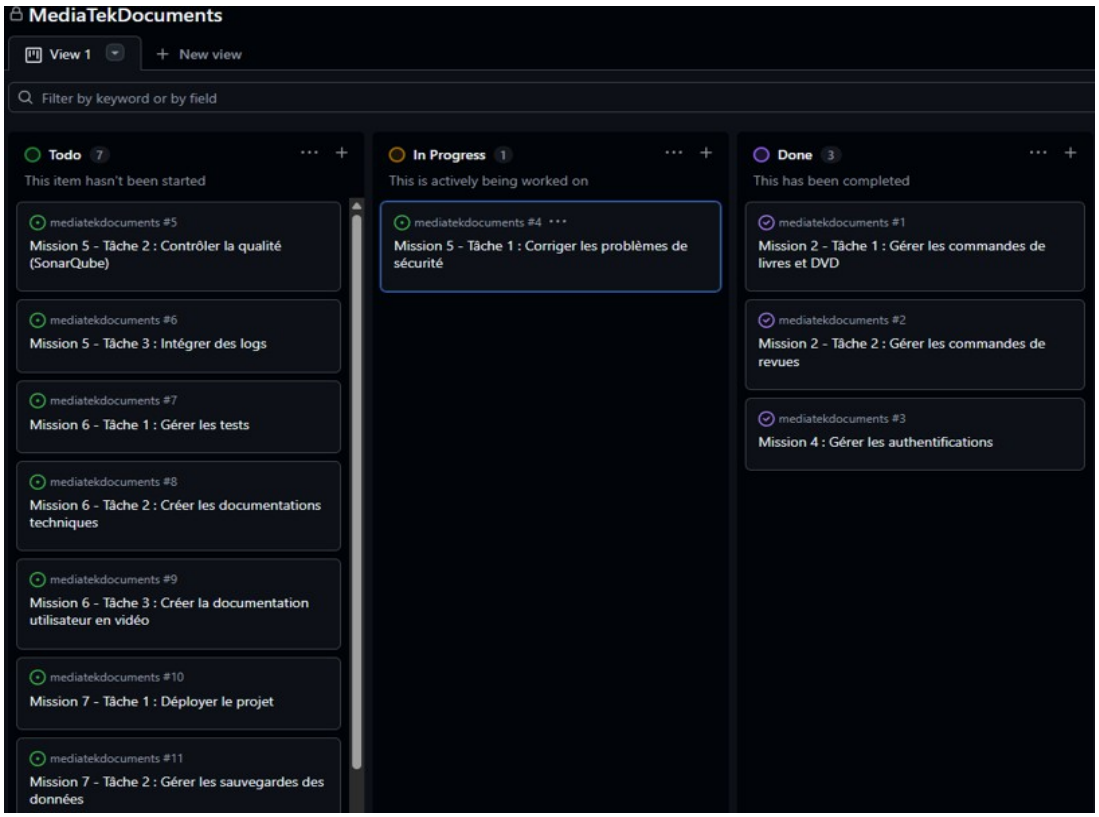
Données de test insérées :

- admin/adminpwd → service Informatique (accès total)
- mdupont/mdp123 → service Comptabilité (accès limité)
- pmartin/mdp456 → service Culture (aucun accès)

Classes impactées :

- Utilisateur.cs — nouvelle classe modélisant un utilisateur
- Access.cs — ajout de la méthode GetUtilisateur()
- FrmMediatekController.cs — ajout de la méthode GetUtilisateur()
- FrmAuthentification.cs — nouvelle fenêtre d'authentification
- FrmMediatek.cs — modification du constructeur pour accepter l'utilisateur et gérer les accès via GererAcces()
- Program.cs — modification pour démarrer sur FrmAuthentification
- MyAccessBDD.php — ajout de la requête selectUtilisateur()



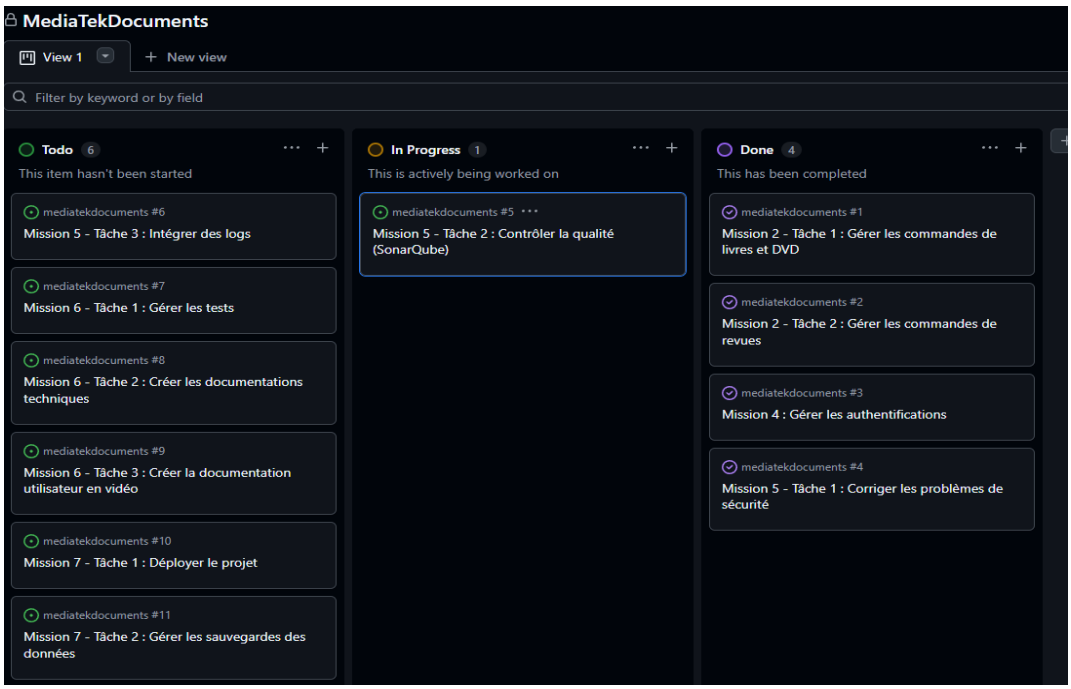


MISSION 5 — Assurer la sécurité, la qualité et intégrer des logs
Tâche 1 — Corriger les problèmes de sécurité

Demande : Corriger 2 problèmes de sécurité : (1) le couple login/mdp de l'API était en dur dans le code, (2) l'API ne retournait pas d'erreur sur une requête vide.

Corrections apportées :

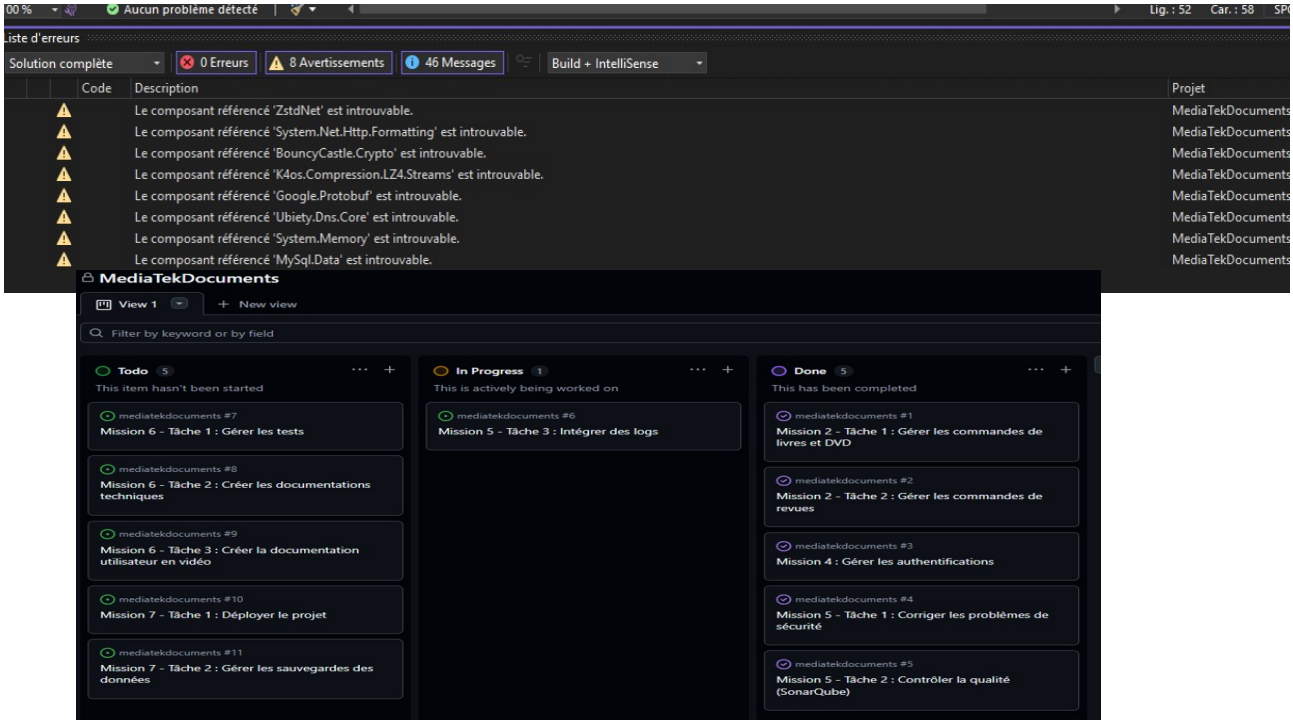
- Problème 1 : déplacement du login/mdp dans App.config via la section appSettings, récupéré dynamiquement dans Access.cs via ConfigurationManager.AppSettings
- Problème 2 : ajout d'une règle dans .htaccess pour retourner une erreur 400 sur requête vide



Tâche 2 — Contrôler la qualité

Demande : Vérifier et corriger les problèmes relevés par SonarQube for IDE dans le code ajouté.

Résultat : Aucun problème bloquant relevé par SonarQube dans le code ajouté. Les avertissements présents concernent des composants tiers non modifiables.



Tâche 3 — Intégrer des logs

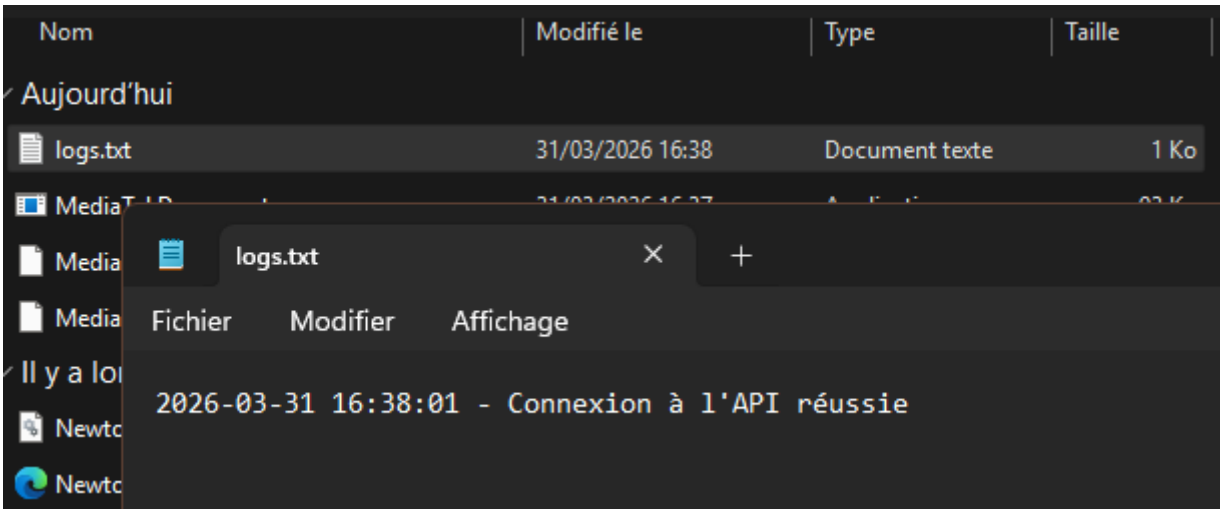
Demande : Ajouter des logs dans la classe Access pour tracer les événements importants.

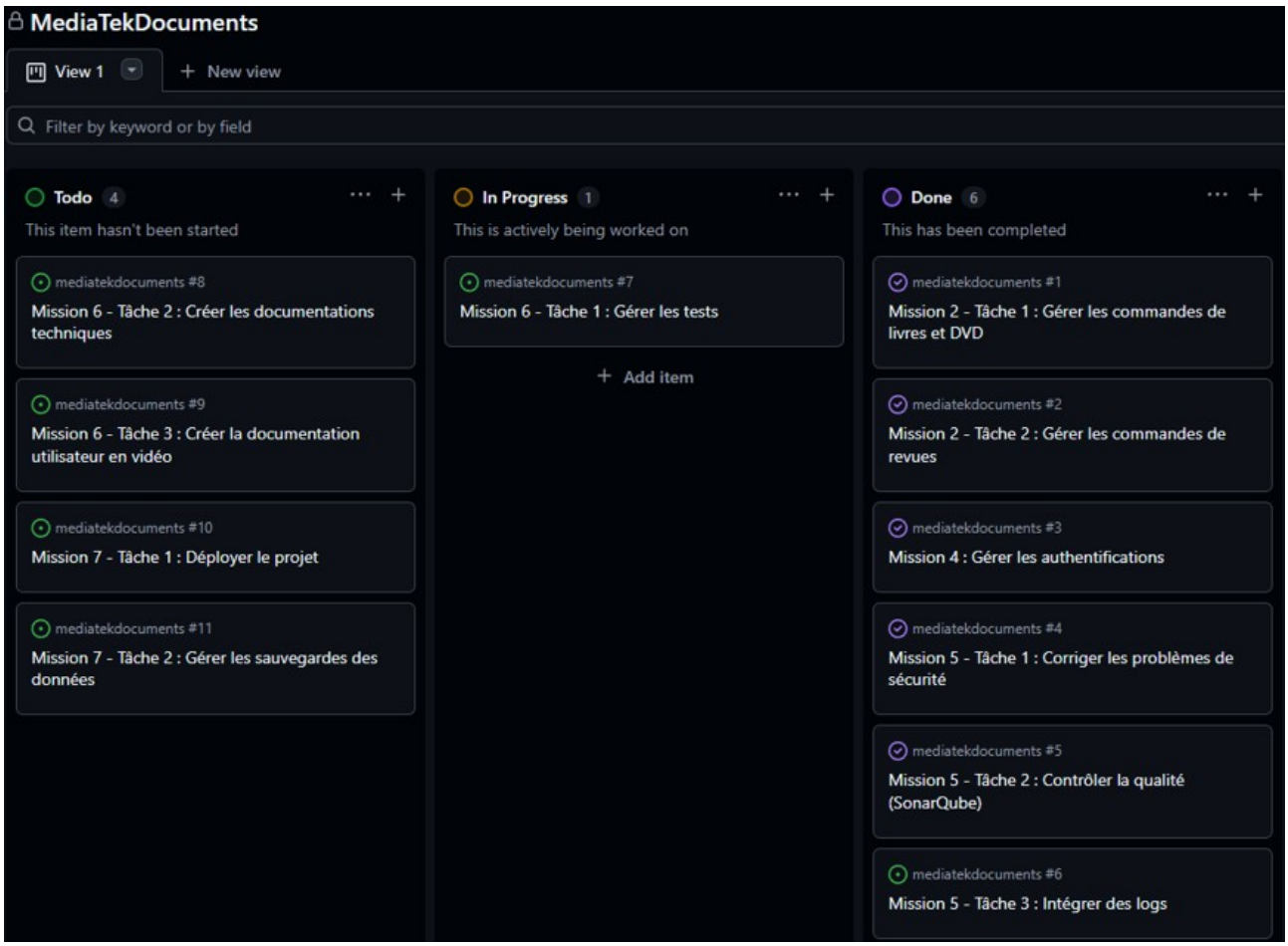
Code de configuration des logs :

csharp

```csharp
private static void log(string message)
{
    string path = "logs.txt";
    using (StreamWriter sw = File.AppendText(path))
    {
        sw.WriteLine(DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss") + " - " + message);
    }
}
```

Exemple de log généré : 2026-03-31 16:38:01 - Connexion à l'API réussie





MISSION 6 — Tester et documenter

Tâche 1 — Gérer les tests

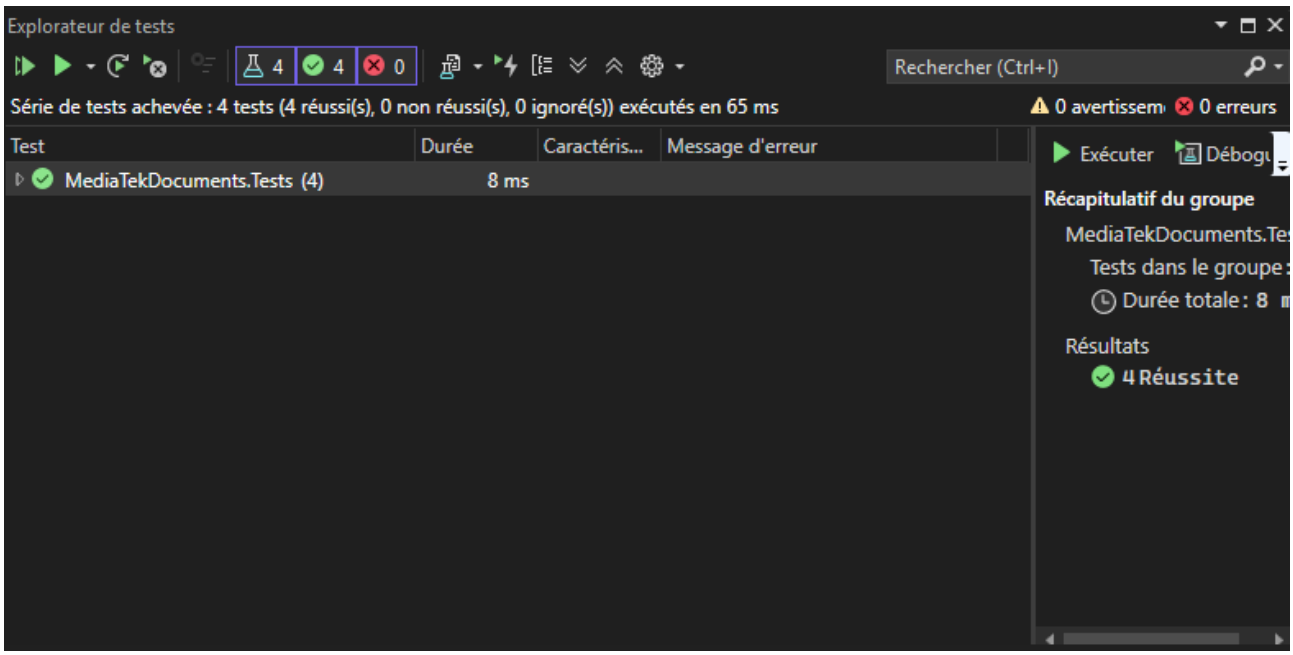
Demande : Écrire les tests unitaires sur les classes du package Model et construire une collection de tests dans Postman pour l'API.

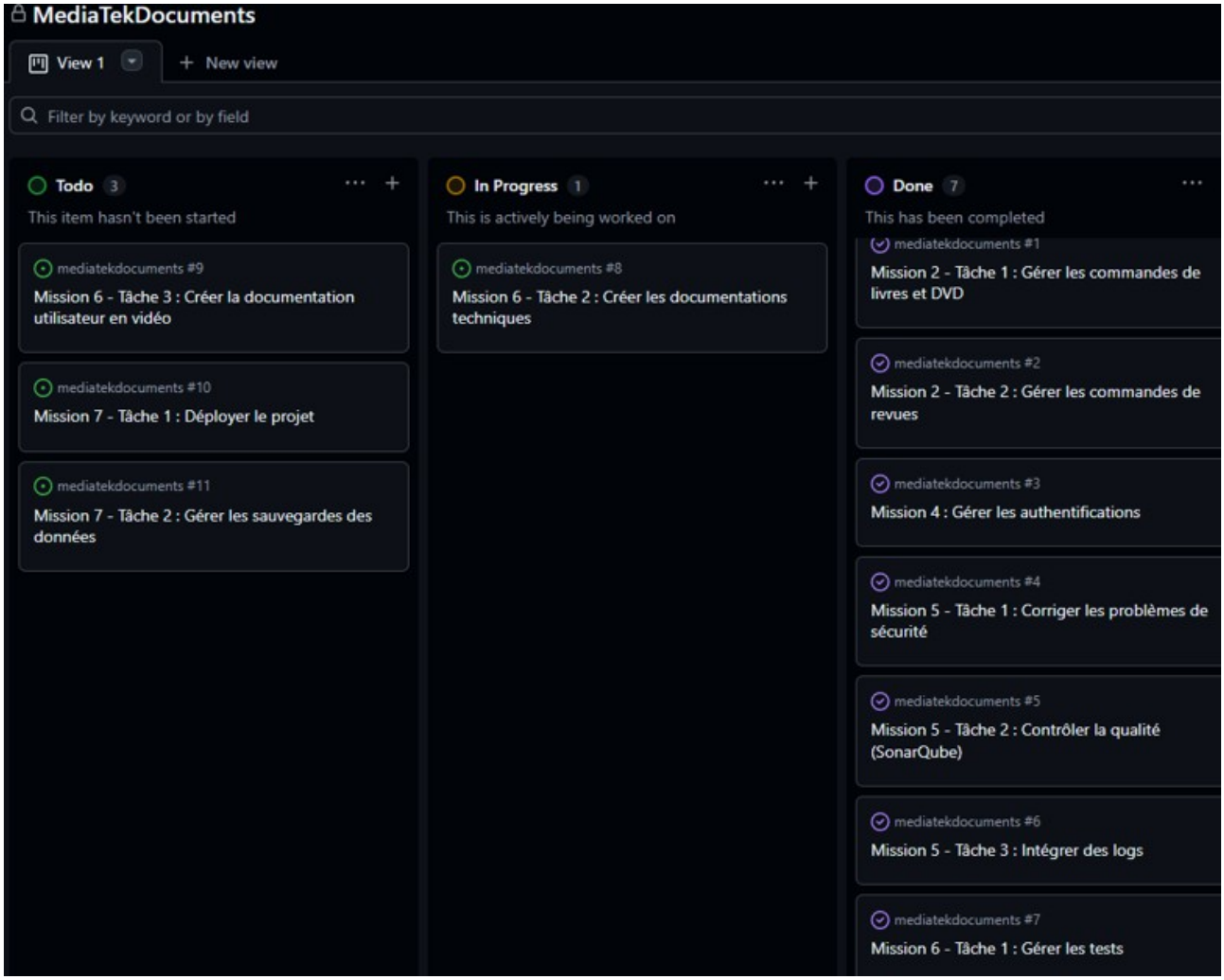
Tests unitaires réalisés : Un projet de tests `MediaTekDocuments.Tests` a été créé. La méthode `ParutionDansAbonnement` a été testée avec 4 cas :

- Date de parution dans l'abonnement → retourne vrai
- Date de parution avant l'abonnement → retourne faux
- Date de parution après l'abonnement → retourne faux
- Date de parution égale à la date de début → retourne vrai

Exemple de code de test :

```
[TestMethod]
public void ParutionDansAbonnement_DateDedans_RetourneVrai()
{
    DateTime dateCommande = new DateTime(2026, 1, 1);
    DateTime dateFin = new DateTime(2026, 12, 31);
    DateTime dateParution = new DateTime(2026, 6, 15);
    Assert.IsTrue(ParutionDansAbonnement(dateCommande, dateFin, dateParution));
}
```

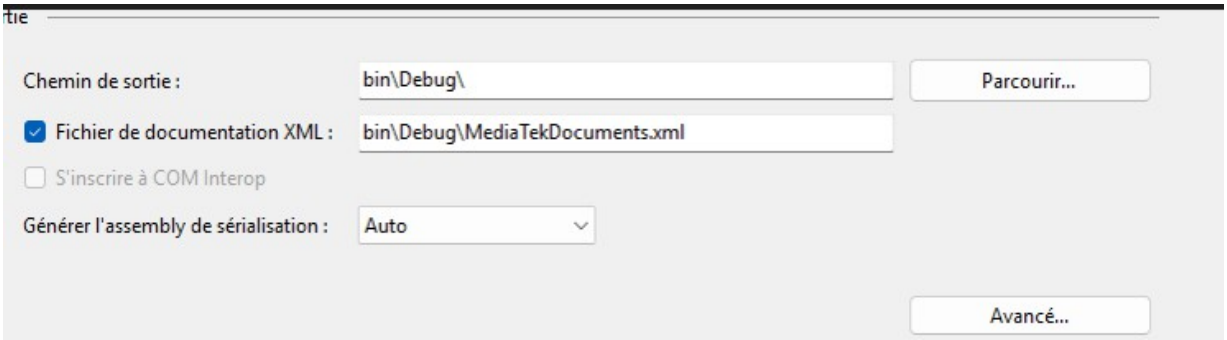


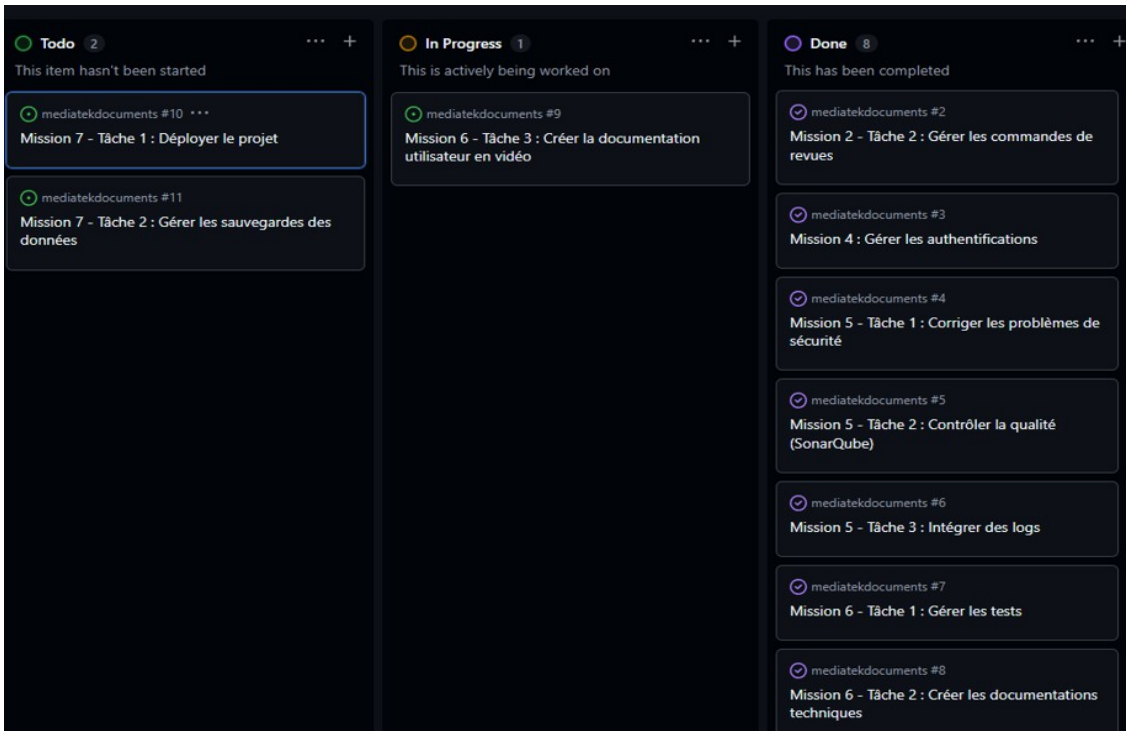


Tâche 2 — Créer les documentations techniques

Demande : Générer la documentation technique de l'application C# via Visual Studio.

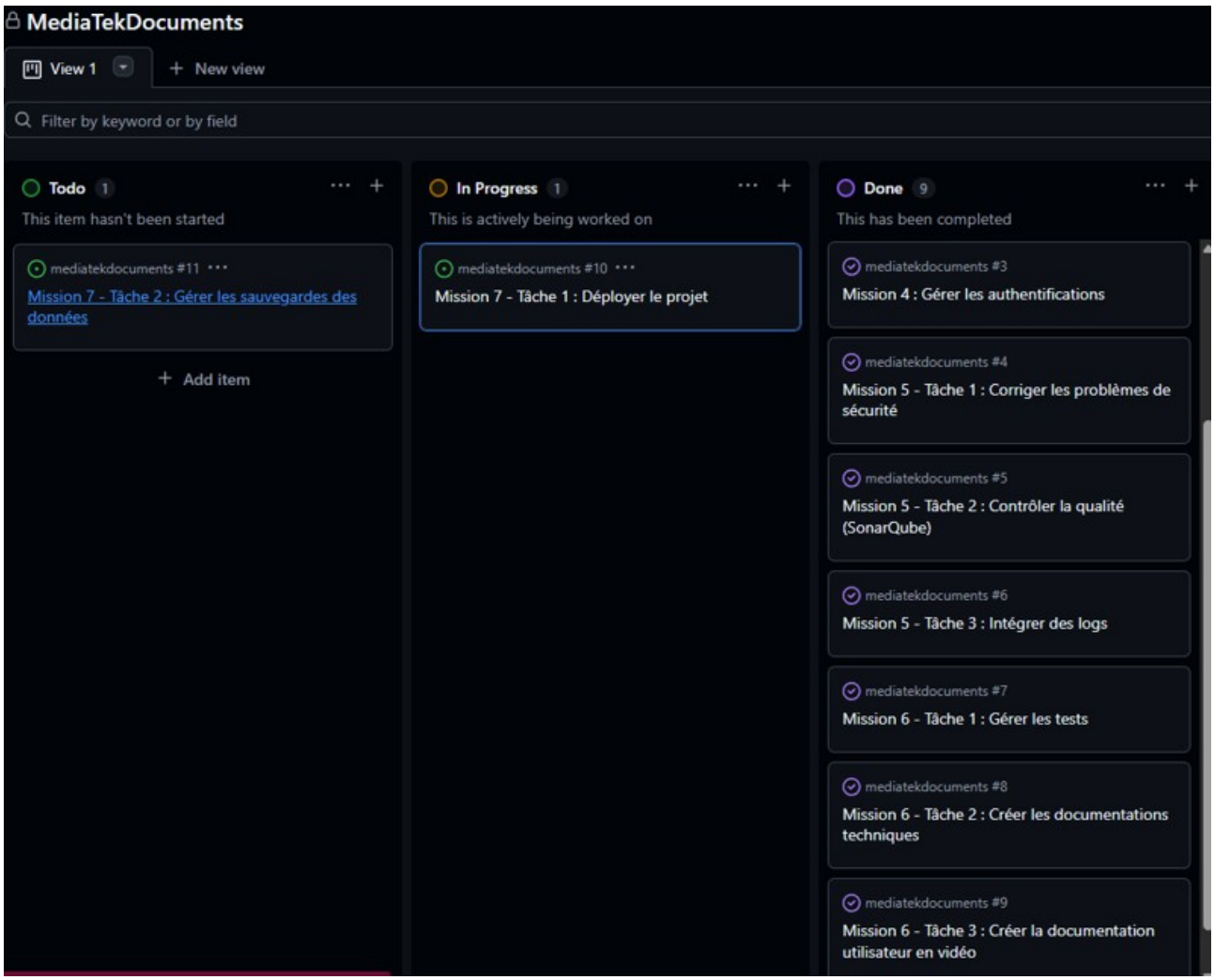
Résultat : La documentation XML a été générée en cochant "Fichier de documentation XML" dans les propriétés du projet Build. Elle est disponible dans le dépôt GitHub.





Tâche 3 — Documentation utilisateur en vidéo

Demande : Créer une vidéo de 10 minutes maximum présentant toutes les fonctionnalités de l'application.


MediaTekDocuments
View 1
New view
Filter by keyword or by field
Todo 1
This item hasn't been started
mediatekdocuments #11
Mission 7 - Tâche 2 : Gérer les sauvegardes des données
Add item
In Progress 1
This is actively being worked on
mediatekdocuments #10
Mission 7 - Tâche 1 : Déployer le projet
Done 9
This has been completed
mediatekdocuments #3
Mission 4 : Gérer les authentifications
mediatekdocuments #4
Mission 5 - Tâche 1 : Corriger les problèmes de sécurité
mediatekdocuments #5
Mission 5 - Tâche 2 : Contrôler la qualité (SonarQube)
mediatekdocuments #6
Mission 5 - Tâche 3 : Intégrer des logs
mediatekdocuments #7
Mission 6 - Tâche 1 : Gérer les tests
mediatekdocuments #8
Mission 6 - Tâche 2 : Créer les documentations techniques
mediatekdocuments #9
Mission 6 - Tâche 3 : Créer la documentation utilisateur en vidéo